

Technical whitepaper library

The five-paper technical case for Onam — find the path, price it, and the architecture, trust and compliance beneath it.

[Attack paths](#)[FAIR pricing](#)[Architecture](#)[Security & trust](#)[Compliance](#)

CONTENTS

Five papers, one argument.

The technical case for Onam, end to end: how a verified attack path is found, priced, architected, secured, and mapped to compliance. Each paper stands alone; together they are the full reasoning behind "one graph, priced risk."

01 How Onam finds the attack path

Graph reasoning, ~25 edge derivers, BFS entry → crown-jewel, five-domain verification, MITRE ATT&CK, choke points.

02 Cloud risk in dollars (FAIR)

The FAIR decomposition, named external inputs (IBM 2024), and why a defensible dollar figure beats a proprietary score.

03 One graph, one data model

The canonical finding contract, the per-tenant property graph, write-many/read-one, and extensibility as the moat.

04 Security & trust

Least-privilege connection, retained-vs-never-retained, three-layer tenant isolation, certifications.

05 Compliance, mapped once

One rule → many controls, rules-as-code that auto-extends frameworks, and gap → priced path.

How to read this. Papers follow the pipeline in order — find (01) → price (02) → the architecture that makes both possible (03) → the trust model behind the access it needs (04) → how the same evaluation serves compliance (05). Every claim traces to Onam's documented design.

How Onam finds the attack path — and proves it before it alerts you

A reproducible method for turning millions of cloud facts into a short list of verified, priced routes to your crown jewels.

[Graph reasoning](#)[5-domain verification](#)[MITRE ATT&CK](#)[Choke points](#)[FAIR pricing](#)

EXECUTIVE SUMMARY

A finding is noise. A verified path is a decision.

Every cloud scanner produces thousands of findings. The hard problem is not detection — it is knowing which handful of them combine into a route an attacker could actually walk to something that matters, and what that route is worth in dollars. Onam treats that as a graph-reasoning problem, solved the same way every time.

This paper describes the method end to end: how Onam ingests seven clouds read-only, normalizes everything into one property graph, derives the relationships an attacker would exploit, searches for routes from internet-reachable entry points to crown jewels, and — critically — **verifies every edge across five independent security domains before it is ever called a confirmed path**. Each hop is mapped to a MITRE ATT&CK technique; the full set of paths is reduced to its top five choke points; and the resulting exposure is priced with the FAIR model so the output is a board-ready number, not a severity label.

The claim, precisely. Onam does not promise to block attacks at runtime. It promises something more useful for prioritization: a **reproducible, verified, and priced** map of the routes that exist in your cloud right now — computed from read-only telemetry, with a deterministic identity for every finding so results are stable across rescans.

01 The problem with a list

A modern multi-cloud estate generates findings faster than any team can trace them. The industry's default answer is severity: sort by Critical, work down the list. But severity is a property of a finding *in isolation*. It cannot tell you that a "medium" public endpoint, a "low" over-permissive role, and an unencrypted data store are — together — a single four-hop route to your customer records.

Attackers do not read your list. They chain. A route is only as strong as its weakest link, and the link that matters is rarely the one with the scariest CVSS score. To prioritize like a defender who thinks like an attacker, you need to reason over the *relationships* between findings, not the findings themselves.

FIGURE 1 — THE SHIFT

SEVERITY LIST

CRIT · unrelated finding
HIGH · unrelated finding
MED · public endpoint — on a path
LOW · over-priv role — on a path

the two that matter are buried at the bottom

VERIFIED PATH



The same four findings, re-framed. Severity scatters them; the graph connects them into one prioritizable route with a single highest-leverage fix. Illustrative.

02 Ingestion — read-only, agentless, seven clouds

The method starts with telemetry that is safe to collect. Onam connects to each cloud with the lowest standing privilege that permits a security read, and stores no long-lived credentials — only a reference to the role or principal, exchanged for short-lived tokens at scan time.

Cloud	Connection	Privilege posture
AWS	Cross-account IAM role via <code>sts:AssumeRole</code> , unique External ID per tenant	SecurityAudit + ReadOnlyAccess; STS creds $\leq$ 60 min
Azure	Entra service principal	Reader at subscription scope + read-only Graph
Google Cloud	Service account or keyless Workload Identity Federation	6 read-only roles (viewer, securityReviewer ...)
OCI	Dedicated IAM user, API signing key	Policy limited to <code>inspect</code> + <code>read</code>
Alibaba / IBM	RAM read-only · Service ID (Viewer + Reader)	Read-only; API key → short-lived Bearer
Kubernetes	Read-only ServiceAccount, agentless	ClusterRole <code>get / list / watch</code>

Each connection runs a four-stage pipeline — **enumeration** → **enrichment** → **normalization** → **drift** — writing an asset inventory and, crucially, an *asset-relationship* table. The relationships are what make the next step possible.

03 One data model — the property graph

Every engine, for every cloud, emits the **same standardized finding contract** into one canonical findings store. There are no per-engine report tables to reconcile. On top of that store, Onam builds a **Neo4j property graph per tenant**: assets are nodes; typed relationships are edges.

- ◆ **Containment** — an account contains a VPC contains a subnet contains an instance.
- ◆ **Attachment** — a role, policy, security group, or volume attached to a resource.
- ◆ **IAM trust** — who can assume, impersonate, or escalate to whom.
- ◆ **Network reachability** — what can actually reach what, across routes and peering.

Normalization is what lets one graph span seven clouds. A resource's identity is canonicalized to `resource_uid` (rather than an ARN vs OCID), its owner to `account_id` (vs subscription vs project). Every finding carries a `finding_id` that is a **deterministic hash** — the same issue produces the same identifier on every rescan, so paths are stable and remediation can be tracked. A `tenant_id` is a not-null key on every row.

Why a graph, honestly. A security graph is now table stakes — every serious CNAPP markets one. Onam's difference is not that the graph exists; it is what happens to an edge *before* it's confirmed (Section 05), and what the finished path is priced at (Section 07).

04 Deriving the edges an attacker would use

Raw relationships describe how the cloud is built. Attack-path reasoning needs edges that describe how it could be *abused*. Onam loads assets, relationships, and failing findings into the graph, then runs roughly **25 catalog-driven edge derivers** — each one encodes a way an attacker moves: assume-this-role, use-this-key, reach-this-port, read-this-secret, escalate-through-this-policy.

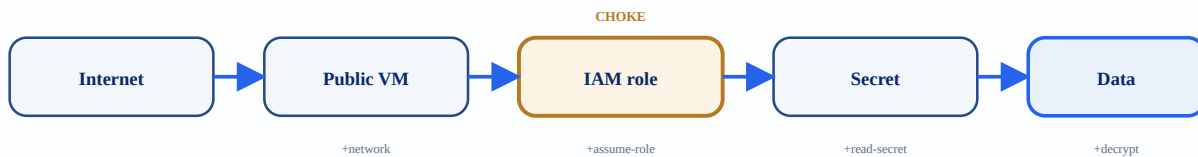
With the abuse-edges in place, the graph is classified at both ends of any route:

- A Entry points** — assets that are internet-reachable, the places an external attacker can actually start.
- B Crown jewels** — sensitive data stores and administrative identities, the places an attacker wants to end.

05 Search — BFS from entry to crown jewel

Onam performs a breadth-first search from every entry point toward every crown jewel, **accumulating attacker capability at each hop**. Reaching a role that can read a secret changes what is reachable next; the search carries that state forward, so a path is evaluated the way an intruder would experience it — as a growing set of capabilities, not a static list of nodes.

FIGURE 2 — CAPABILITY ACCUMULATES ALONG THE WALK



Each hop adds capability (labels under the nodes). The search reasons over the accumulated set, not each node alone. Illustrative.

Verification — five domains before "confirmed"

This is the step that separates a plausible path from a real one. Before any edge is allowed to stand as part of a **CONFIRMED** path, it is verified across **five independent security domains** — never on a single signal. A relationship that looks exploitable from IAM alone but is contradicted by network reachability does not survive. The effect is a dramatically shorter, higher-confidence path list: the routes that pass are the ones worth a human's attention.

06 Mapping to MITRE ATT&CK, then to choke points

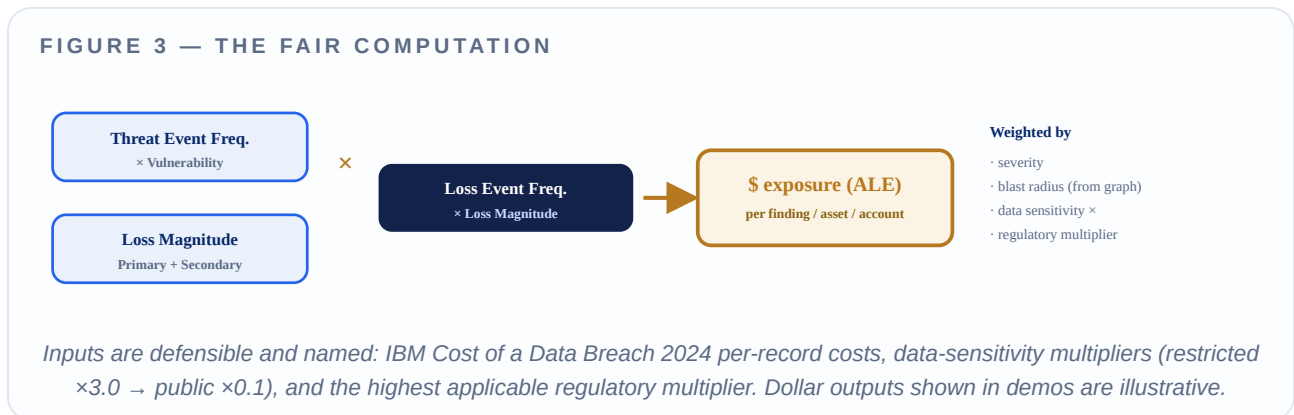
Every hop on a confirmed path is annotated with the **MITRE ATT&CK technique** it represents, producing a technique chain that a threat-informed defender can reason about directly — for example credential access via T1552.005 (cloud instance metadata) leading to valid-accounts use via T1078.004 and data access via T1530. Onam maps to ATT&CK; it does not claim exhaustive coverage of the framework.

Across all confirmed paths, a few nodes recur far more than others. Onam computes the **top five choke points** — single resources whose remediation severs the greatest number of distinct paths — and tells you to triage those first. One well-chosen fix at a choke point can neutralize dozens of routes at once, which is why choke-point ranking, not finding count, drives the work queue.

Severity	Remediation window	Lifecycle
Critical	7 days	Auto-resolves on next PASS; auto-closes after 30 days unseen. Deterministic IDs keep history intact across rescans.
High	30 days	
Medium	90 days	
Low	next cycle	

07 Pricing the path — FAIR as the fourth layer

A verified, ATT&CK-mapped path is still a technical artifact. To make it a business decision, Onam prices it with **FAIR** (Factor Analysis of Information Risk), the open standard for quantifying risk in financial terms.



The result is a dollar exposure per finding, asset, and account — weighted by severity and by the *blast radius the graph already computed*. That is the number a CISO can take to a board, and it is derived, not asserted.

08 End to end — an illustrative walk

Putting the method together on a single representative route:

- 1 Ingest.** Read-only connect surfaces a public-facing VM (entry point) and a customer-records store (crown jewel).
- 2 Derive.** Edge derivers find: the VM's instance role can be assumed; that role can read a secret; the secret decrypts access to the records store.
- 3 Search.** BFS connects internet → VM → role → secret → records, carrying capability forward at each hop.
- 4 Verify.** The four edges are checked across five domains; all hold. The path is marked CONFIRMED.
- 5 Map & rank.** Each hop gets its ATT&CK technique; the *IAM role* is identified as a choke point shared by many other paths.
- 6 Price.** FAIR values the exposure using the records' sensitivity and count. The output: fix one role, cut the most dollars.

09 What this method does not do

Posture, detection, and priced reasoning — not inline blocking. Onam is read-only and agentless by design. It computes and prices the routes that exist; it does not sit in the data path and stop traffic at runtime. Cloud-detection signals come from audit logs (e.g. CloudTrail, Azure Monitor, Kubernetes API), not from an inline enforcement agent. For live blocking, Onam is complementary to a runtime enforcement tool — its role is to tell you which few things to fix so the runtime layer has far less to catch.

We state this plainly because the value of the method depends on trusting it. A path list you can reproduce, verify, and price is worth more than a longer list of things that might be true.

10 Summary

Onam converts read-only, multi-cloud telemetry into one property graph; derives the edges an attacker would abuse; searches for routes from real entry points to real crown jewels; and — before calling anything confirmed — verifies each edge across five independent domains. It maps every hop to MITRE ATT&CK, reduces the whole path set to its five highest-leverage choke points, and prices the exposure with FAIR against named, defensible inputs. The output is short, verified, ranked, and denominated in dollars.

See it on your own cloud. The entire method runs read-only. A connect takes minutes and nothing is installed on a workload — you see your own entry points, paths, choke points, and priced exposure, computed the way this paper describes.

Cloud risk in dollars

How Onam prices a verified attack path with FAIR — using named, external inputs a board can defend.

FAIR / ALE

IBM 2024 costs

Sensitivity ×

Regulatory ×

Exposure roll-up

EXECUTIVE SUMMARY

Severity tells you what is broken. Dollars tell you what to fix first.

Security teams are drowning in "criticals." Boards are not asking how many — they are asking how much. This paper explains how Onam turns a verified attack path into a defensible financial figure using FAIR, the open standard for risk quantification, so the conversation moves from a spreadsheet of severities to a single number a board can act on.

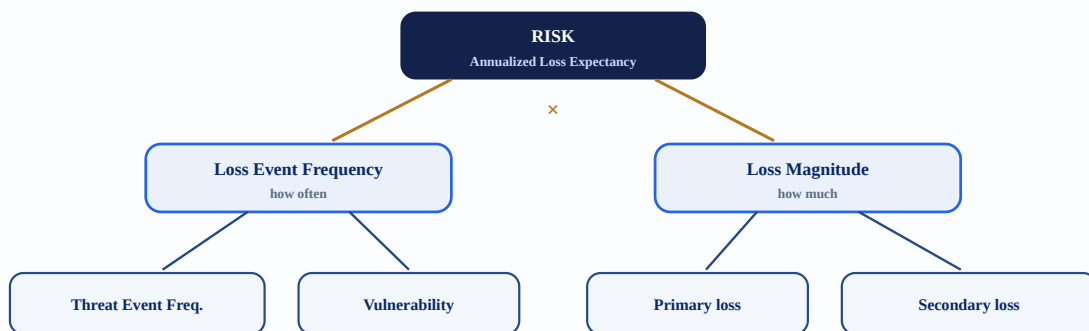
FAIR — Factor Analysis of Information Risk — is a published, peer-reviewed methodology maintained as an open standard. It decomposes risk into estimable factors and produces an Annualized Loss Expectancy (ALE) in currency. Onam implements FAIR as the fourth layer of its pipeline: after a path is found, verified, and mapped to MITRE ATT&CK, it is priced. The pricing is not a marketing gauge invented in-house; it is a recognized model fed with named, external inputs.

Why this is the wedge. A security graph and attack paths are now common across the CNAPP market. A defensible dollar figure — derived from a published model and named external cost data, attached to a specific verified path — is not. It is the difference between "we have 200 criticals" and "this one over-privileged role carries the largest single slice of our quantified exposure."

01 The FAIR model, briefly

FAIR expresses risk as the product of how often a loss happens and how big it is when it does:

FIGURE 1 — FAIR DECOMPOSITION



Onam computes each leaf factor from graph-derived evidence rather than a workshop guess: frequency and vulnerability from reachability and control state; magnitude from data sensitivity and record counts.

The strength of FAIR is that each factor is *estimable and inspectable*. When Onam prices a path, a reviewer can open the calculation and see which input drove the number — not a black-box score between 0 and 100.

02 The inputs — named, external, defensible

A dollar figure is only as credible as the numbers behind it. Onam deliberately sources its magnitude inputs from a recognized external benchmark and applies transparent multipliers.

Input	Source / basis	Effect
Per-record breach cost	IBM <i>Cost of a Data Breach 2024</i> — by industry	Healthcare ≈ \$10.93/record ... default ≈ \$4.45/record
Data-sensitivity multiplier	Onam data classification (DSPM)	Restricted ×3.0 → Confidential → Internal → Public ×0.1
Regulatory multiplier	Applicable regime (GDPR, etc.)	Highest applicable applied (e.g. GDPR ×1.5)
Blast radius	The property graph	Weights exposure by how much the path can actually reach
Severity	Finding evaluation	Weights likelihood contribution

Defensible by construction. Every magnitude figure traces to IBM's published per-record cost multiplied by a sensitivity factor and, where relevant, a regulatory factor. An auditor or a skeptical board member can follow the arithmetic. That traceability is the point.

03 From a path to a price

- 1 Locate the loss.** A verified path ends at a crown jewel — a data store or admin identity. Onam reads the store's classification and record count (labels and locations only; contents are never retained).
- 2 Estimate frequency.** Loss Event Frequency is shaped by how reachable the entry is and how many controls the path defeats — evidence the graph already holds.
- 3 Estimate magnitude.** Record count × IBM per-record cost × sensitivity × regulatory multiplier yields Loss Magnitude.
- 4 Roll up.** Exposure aggregates per finding, per asset, and per account — so the same model answers "what is this path worth" and "what is this account's total exposure."

04 Why a dollar beats a score

Competing platforms prioritize with proprietary scales — toxic-combination flags, 0–99 business-impact scores, Critical-to-Low risk levels. Each is useful internally, but none is a currency figure a board recognizes, and none is portable across a risk register that already speaks in dollars.

FIGURE 2 — THE SAME EXPOSURE, TWO LANGUAGES

SCORE (proprietary)

Business impact: 87 / 100

Board question it cannot answer: "so how much?"

DOLLARS (FAIR)

Annualized exposure on this path: \$X.XM

Slots directly into the enterprise risk register.

Figures shown in demos are illustrative, not benchmarks.

A score ranks within one tool. A FAIR dollar figure travels — into the risk register, the board pack, and the cyber-insurance conversation.

05 Prioritization changes when risk is priced

Once every path carries a dollar figure, the work queue re-sorts itself around **exposure reduced per fix**. A choke point that sits on many priced paths rises to the top even if its own finding severity is modest; an isolated "critical" with no path to anything valuable falls. This is the practical payoff: teams spend their limited remediation budget where it removes the most quantified risk, and they can prove it afterward — exposure before, exposure after.

The board sentence. "We reduced quantified cloud exposure by \$N this quarter by fixing five choke points" is a sentence FAIR makes possible and a severity list does not.

06 Honesty about a quantified number

A model, not a prophecy. FAIR produces a defensible estimate, not a guaranteed outcome. Its value is in relative prioritization and in giving the board a currency-denominated view — not in predicting the exact cost of a future breach. Onam labels demo dollar figures as illustrative and never presents them as customer benchmarks. The inputs are named so the estimate can be challenged and refined, which is exactly how a credible risk figure should behave.

Two disciplines keep the number trustworthy. First, inputs are external and cited (IBM 2024), so the magnitude side is anchored to published data rather than internal assertion. Second, the frequency side is driven by graph-derived evidence of reachability and control state, so it moves when the environment actually changes — a path that gets harder to walk gets cheaper, automatically, on the next scan.

07 Summary

Onam prices verified attack paths with FAIR, an open and inspectable model, using named external inputs — IBM's 2024 per-record breach costs, data-sensitivity multipliers, and regulatory factors — weighted by the blast radius the graph already computed. The result is an annualized exposure figure per finding, asset, and account that a board recognizes and a risk register can hold. Where competitors stop at a proprietary score, Onam produces a currency figure you can defend, challenge, and track down over time.

See your number. A read-only connect prices your own environment the way this paper describes — every figure traceable to a named input, computed on the same graph that found the path.

One graph, one data model

The architecture beneath Onam — how seven clouds and every capability share one store, one graph, one source of truth.

Canonical finding

Per-tenant graph

Shared writer

Deterministic IDs

Rules as code

EXECUTIVE SUMMARY

Every capability writes to one place. That is the architecture.

Most security platforms are a drawer of tools that each keep their own findings and their own view of the world. Onam is built the opposite way: every engine, for every cloud, writes the same finding contract into one shared store, and everything reasons over one property graph. This paper explains that design and why it is the thing that makes cross-cloud attack paths, priced risk, and painless extensibility possible.

The architectural claim is specific and testable: there are **no per-engine report tables** to reconcile. A gateway reads only from one canonical findings store; a per-tenant graph turns those findings and the assets they touch into nodes and typed edges. Adding a capability does not add a silo — it adds another writer to the same store.

Why it matters to a buyer. Consolidation stops being a slide and becomes a data model.

Posture, identity, data, workloads, SaaS, and runtime detection share one schema, so a finding in one domain can form part of an attack path that ends in another — which is impossible when each tool hoards its own results.

01 Ingestion into a normalized inventory

Each cloud connects read-only and agentless (Whitepaper 01, §02). The pipeline runs four stages — **enumeration** → **enrichment** → **normalization** → **drift** — and writes two things every capability depends on: an asset inventory and an asset-relationship table.

FIGURE 1 — MANY CLOUDS, ONE NORMALIZED STORE



Seven clouds plus SaaS collapse into one normalized identity space, then into one findings store and one relationship table. Everything downstream reads from here.

02 The canonical finding contract

Consolidation lives or dies on the schema. Onam requires every engine to emit the **same standardized finding**, keyed consistently, so one table can hold results from posture, identity, data, workload, code, and SaaS engines without translation.

Field	Canonicalizes	Why it matters
resource_uid	ARN vs OCID vs self-link	One identity for an asset across clouds
account_id	account vs subscription vs project	Ownership normalized across providers
finding_id	deterministic hash	Same issue → same ID on every rescan; stable history
tenant_id	not-null isolation key	Every row is tenant-scoped by construction
provider_metadata	provider-specific detail	Nothing is lost; specifics travel with the finding

Because `finding_id` is deterministic, remediation tracking is reliable: a fixed issue auto-resolves on the next PASS and auto-closes after 30 days unseen, without a human reconciling IDs by hand.

03 The property graph

On top of the findings and relationships, Onam builds a **Neo4j property graph per tenant**. Assets are nodes; edges are typed — containment, attachment, IAM trust, and network reachability — and roughly 25 catalog-driven derivers add the abuse-edges an attacker would use. This is the substrate for attack-path reasoning (Whitepaper 01).

Per-tenant, by design. The graph is not a shared multi-tenant structure with a filter bolted on. Each tenant has its own graph, and a `tenant_id` filter is applied at the database on every query — one of three isolation layers described in Whitepaper 04.

04 The gateway reads one store

A single BFF (backend-for-frontend) gateway serves the product — on the order of 166 endpoints across roughly 60 routers behind a FastAPI service — and it reads **only** from the canonical findings store, never from per-engine tables. That constraint is what keeps the "one view" promise honest: there is no second source of truth to drift.

FIGURE 2 — WRITE MANY, READ ONE



Engines fan in through a shared writer; the gateway fans out from a single store. No per-engine report tables exist to reconcile.

05 Scale without a rebuild

Work parallelizes per account, service, and region, so a fifty-account organization scans in roughly the same wall-clock as one. Onam publishes scan SLO targets by resource count and API latency targets rather than customer benchmarks:

Resources	Scan SLO	API target
< 1,000	15 min	p95 ≤ 500 ms read · attack-graph traversal < 3 s · findings-to-console < 5 min
1k – 10k	60 min	
10k – 100k+	2 – 4 hr	

06 Extensibility — the real moat

The payoff of one data model is what it costs to add the next capability. A new engine does not get a new database, a new API surface, or a new UI silo. It writes the **same finding contract** into the **same store** through the shared writer, keyed by (tenant_id, account_id, rule_id, resource_uid). The gateway and graph pick it up unchanged.

- 1 Rules as code.** The platform loads and evaluates versioned YAML rules; it never hardcodes them. New rules auto-extend every framework they map to.
- 2 New engine, same contract.** A new capability emits the canonical finding; attack-path reasoning and FAIR pricing apply to it for free.
- 3 Custom without forking.** A Rule Builder lets teams add their own rules; custom frameworks map to rule IDs — no code change.

Consolidation, provable. Because every capability shares the schema, "one platform instead of a drawer of tools" is not a claim about the sales bundle — it is a claim about where the data lives. That is the difference this architecture buys.

07 Summary

Onam normalizes seven clouds plus SaaS into one identity space, one canonical findings store, and one per-tenant property graph. A single gateway reads only from that store, so there is no second source of truth. Deterministic finding IDs make history reliable; catalog-driven drivers turn relationships into attack-path edges; and new capabilities extend the platform by writing the same contract rather than adding a silo. The graph is table stakes; the shared data model beneath it is the architecture that makes cross-cloud paths and priced risk possible.

Security & trust

How Onam connects, what it stores, what it never stores, and how tenants stay isolated — the trust case for a read-only platform.

Least privilege

No stored secrets

Metadata not data

Per-tenant isolation

SOC 2 · ISO · PCI

EXECUTIVE SUMMARY

A security tool has to earn more trust than it asks for.

Onam reads your entire cloud. That access is exactly why our own security posture has to be exemplary and legible. This paper documents how Onam connects, what it stores, what it deliberately never stores, and how tenants are isolated — so a security team can grant access with confidence rather than hope.

The design principle is minimality: take the least privilege that permits a security read, hold no long-lived secrets, and retain only the metadata needed to describe a risk — never the data itself. Where a control is a stated certification, it is named; where something is planned rather than achieved, this paper does not claim it.

The short version. Read-only, agentless, short-lived tokens, per-tenant isolation with per-tenant keys, and a retention policy that keeps labels and locations but not contents. The rest of this paper is the detail behind each of those words.

01 Least-privilege connection

Every cloud is connected with the lowest standing privilege that still allows a security read, and no long-lived cloud credential is ever stored — only a reference (a role ARN or principal ID) held in a secrets manager and encrypted with a managed key. At scan time that reference is exchanged for a short-lived token.

Cloud	Mechanism	Privilege
AWS	Cross-account role, unique External ID/tenant	SecurityAudit + ReadOnlyAccess; STS ≤ 60 min
Azure	Entra service principal	Reader + read-only Graph
GCP	Service account or keyless WIF	6 read-only roles
OCI	IAM user + signing key	inspect + read only
Alibaba / IBM	RAM read-only · Service ID	Read-only; key → short-lived Bearer
Kubernetes	Read-only ServiceAccount	get / list / watch

Confused-deputy defense. The AWS connection requires a unique External ID per tenant, so one tenant's role can never be assumed on behalf of another. Tokens expire in an hour or less, and secret material is never written to logs.

02 What Onam never stores

The most important trust control is what does *not* enter the system. Onam keeps the metadata required to describe and price a risk and deliberately discards everything else.

FIGURE 1 — RETAINED VS NEVER RETAINED

RETAINED (metadata only)

- ✓ Asset inventory & relationships
- ✓ Finding metadata (rule, resource ref)
- ✓ Data labels & locations (DSPM)
- ✓ Credential references (not secrets)
- ✓ Config state needed to evaluate rules

NEVER RETAINED

- ✗ Object / file contents
- ✗ Database rows
- ✗ Workload disk / memory (ephemeral)
- ✗ Source code (finding metadata only)
- ✗ Personal data (labels + locations only)

DSPM classifies where sensitive data lives without keeping the data. Workload scanning uses ephemeral snapshots. SecOps retains finding metadata, not source.

03 Three layers of tenant isolation

- 1 Gateway.** An authenticated context is established at the BFF for every request before anything is read.
- 2 Engine.** A feature:action permission is checked at the engine boundary — 27 permissions across 5 seeded roles.
- 3 Database.** A tenant_id filter is applied on every query. Per-tenant encryption keys protect data at rest.

404, not 403. A cross-tenant probe returns "not found," never "forbidden" — so the system does not even confirm the existence of another tenant's resources.

04 Access control & identity

Onam ships role-based access control with five seeded roles mapped to 27 fine-grained feature:action permissions, so operators can grant exactly what a person needs. For enterprise identity, Onam supports **SAML SSO** and **SCIM** provisioning, so account lifecycle follows the customer's identity provider rather than a separate list to maintain.

05 Certifications & attestations

The following are stated as achieved. Onam maps a single finding to many controls, and supports 70+ compliance frameworks out of the box (covered in Whitepaper 05).

Attestation	Scope
SOC 2 Type II	Security, availability, confidentiality controls over a period
ISO 27001:2022	Information security management system
ISO 27017	Cloud-specific security controls
PCI DSS v4.0	Payment-card data security
CSA STAR Level 1	Cloud security self-assessment registry
GDPR	EU data-protection alignment
HIPAA BAA	Business Associate Agreement available

Stated, not aspirational. This paper lists only attestations Onam presents as achieved. Where a control is on a roadmap rather than in place, it is intentionally absent here — trust documentation that overstates is worse than none.

06 An honest boundary

Read-only means read-only. Onam's trust model is built on the fact that it does not sit in your data path. It does not enforce or block at runtime, and it does not hold your data. That is a deliberate limit, not an omission: the smaller the footprint and the less it retains, the smaller the blast radius if Onam itself were ever compromised. For live inline enforcement, Onam is complementary to a runtime tool — and its own posture is easier to trust precisely because it asks for so little.

07 Data handling at a glance

Dimension	Control
Cloud credentials	References only, KMS-encrypted; tokens $\leq$ 1 hr; never logged
Data contents	Never retained — labels & locations only
Workload access	Agentless; ephemeral snapshots, not persistent disk copies
Source code	Finding metadata retained; code not stored
Tenant isolation	Gateway → engine permission → DB <code>tenant_id</code> ; per-tenant keys
Identity	RBAC (5 roles / 27 perms) · SAML SSO · SCIM

08 Summary

Onam takes the least privilege that permits a security read, stores no long-lived cloud secrets, and retains metadata rather than data — labels and locations, not contents. Three isolation layers and per-tenant encryption keep tenants apart, with cross-tenant probes returning 404. The platform is backed by SOC 2 Type II, ISO 27001:2022 and 27017, PCI DSS v4.0, CSA STAR L1, GDPR alignment, and a HIPAA BAA. The trust case rests on a simple idea: a tool that reads everything should ask for as little as possible and keep even less.

Due-diligence ready. This paper doubles as the basis for a security questionnaire response. A read-only trial lets your team verify the connection privileges and retention behavior directly.

Compliance, mapped once

How Onam evaluates a control once and reports it against 70+ frameworks — and connects every gap to a priced attack path.

One rule → many controls

Rules as code

70+ frameworks

Custom frameworks

Gap → priced path

EXECUTIVE SUMMARY

Evaluate the control once. Report it against every framework.

Compliance work is mostly duplication: the same control, re-checked and re-evidenced for PCI, then SOC 2, then ISO, then a customer questionnaire. Onam removes the duplication by decoupling the check from the framework. Rules are evaluated once as code; a single finding maps to many controls across 70+ frameworks at once.

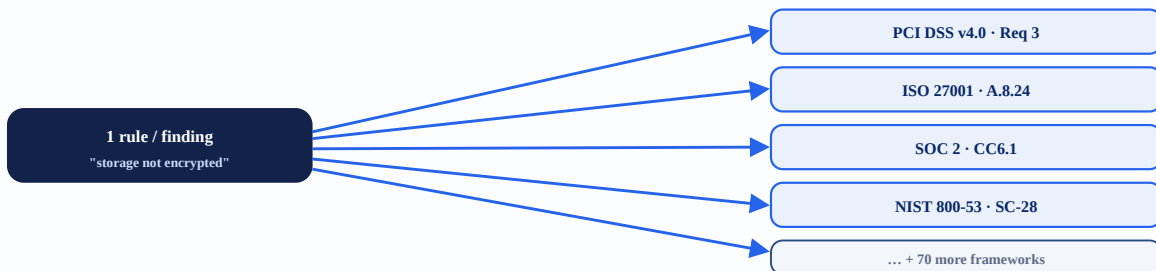
This paper explains the mapping model, why rules-as-code makes framework coverage extend automatically, and how compliance and attack-path reasoning reinforce each other — so a control gap is not just a failed check but a node on a priced path.

An honest framing. Framework *count* is a coverage feature, not a differentiator on its own — several platforms publish large numbers. Onam's distinct value is the *architecture* that produces the mapping (one rule → many controls, auto-extending) and the ability to connect a compliance gap to a priced attack path. This paper leads with the mechanism, not the number.

01 One finding, many controls

A framework is, at bottom, a set of controls. Many controls across many frameworks ask for the same underlying thing — "encrypt data at rest," "restrict public access," "enforce MFA." Onam evaluates that underlying condition once and maps the result to every control it satisfies or violates.

FIGURE 1 — THE MAPPING FAN-OUT



One evaluated condition satisfies (or fails) many controls at once. Fix it once; the status updates everywhere it is mapped. Control IDs are illustrative of the mapping shape.

02 Rules as code — why coverage extends itself

Onam's rules are versioned YAML the platform loads and evaluates; nothing is hardcoded. This is what makes framework coverage compound rather than accumulate by hand. When a new rule is added, it automatically extends every framework whose controls it maps to. When a new framework is onboarded, its controls map to existing rule IDs — no new checks to write.

- 1 Add a rule.** A new YAML rule ships; every mapped framework gains coverage for the condition it checks.
- 2 Add a framework.** A new framework's controls map to existing rule IDs; coverage appears without new detection code.
- 3 Add a custom framework.** A Rule Builder and custom-framework mapping let a team model an internal standard against the same rule library.

The compounding effect. Because checks and frameworks are decoupled, work done for one obligation pays down others automatically. A rule written to satisfy an internal standard may also close a PCI and an ISO control the same day — without anyone re-authoring the check.

03 Frameworks supported

Onam supports 70+ frameworks out of the box across regulatory, industry, and best-practice standards — for example PCI DSS v4.0, SOC 2, ISO 27001/27017, HIPAA/HITRUST, GDPR, NIST 800-53, and CIS benchmarks — with custom frameworks mappable to rule IDs. Coverage is delivered as continuous evaluation, not a point-in-time audit export.

04 Compliance meets the attack path

A failed control is a compliance fact. The same failing condition is also a node in the property graph — which means Onam can tell you not just *that* a control failed, but whether its failure lies on a verified, priced route to a crown jewel. That reframes remediation: two "failures" of equal audit weight are not equal if one sits on a choke point and the other reaches nothing.

FIGURE 2 — THE SAME GAP, TWO LENSES

COMPLIANCE LENS

Control X: FAIL

one row in an audit report

RISK LENS

Same gap = choke point on a \$X.XM path

fix this one first — illustrative figure

Compliance says what is out of policy; the graph says which out-of-policy items an attacker can actually exploit — and prices them. Same finding, two questions answered.

Audit and risk from one evaluation. The auditor gets continuous control status; the security team gets the same gaps ranked by priced exposure. Neither needs a separate scan, because both read the same findings store.

05 What framework count does and does not prove

Coverage breadth is necessary, not sufficient. A large framework number tells a buyer they will not be missing a standard they need; it does not by itself prove depth of mapping or usefulness of the result. Onam presents 70+ frameworks honestly as broad out-of-the-box coverage, and points to the mechanism — one rule to many controls, auto-extending, connected to priced paths — as the reason the coverage is maintainable and actionable rather than a static checklist. Buyers comparing vendors should weigh mapping architecture and risk-linkage, not the headline count alone.

06 Summary

Onam evaluates security conditions once as versioned rules and maps each result to many controls across 70+ frameworks, so a single fix updates compliance status everywhere it applies. Rules-as-code makes coverage compound: new rules extend every mapped framework, and new frameworks map to existing rules without new detection code. Because the same findings feed the property graph, a control gap can be seen as what it really is — a node that may sit on a priced attack path — letting teams satisfy auditors and reduce real risk from one evaluation. The framework count is the floor; the mapping architecture and risk-linkage are the value.

Map your obligations once. A read-only connect shows your current control status across every mapped framework, with the gaps that sit on priced paths flagged first.